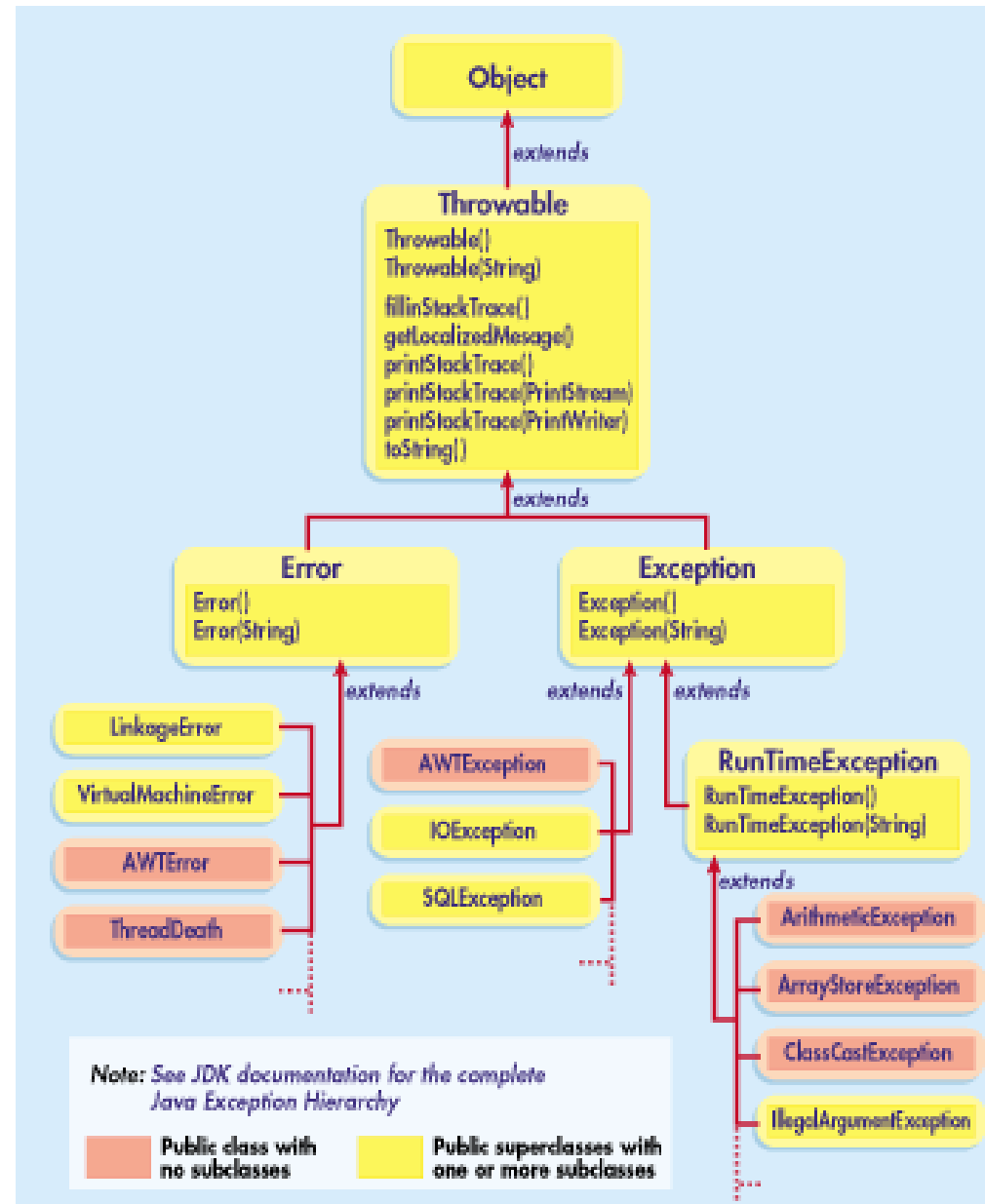


Exception Handling

Exceptions and Errors

- When a program encounters an unexpected termination or fault, it is called an exception
- When we try and divide by 0 we terminate abnormally.
- Exception handling gives us another opportunity to recover from the abnormality.
- Sometimes we might encounter situations from which we cannot recover like Outofmemory. These are considered as errors.

Hierarchy



Hierarchy

- Throwable class is a super class of all exception and errors.
- Exceptions has a special subclass, the RuntimeException
- A user defined exception should be a subclass of the exception class

Types of exceptions

- Exceptions which are checked for during compile time are called checked exceptions.

Example: SQLException or any userdefined exception extending the Exception class

- Exceptions which are not checked for during compile time are called unchecked exception.

Example: NullPointerException or any class extending the RuntimeException class.

- All the checked exceptions must be handled in the program.
- The exceptions raised, if not handled will be handled by the Java Virtual Machine. The Virtual machine will print the stack trace of the exception indicating the stack of exception and the line where it was caused.

Types of exceptions

- **ArithmeticException:** `int a=50/0;//ArithmeticException`
- **NullPointerException:**
 - `String s=null; System.out.println(s.length());//NullPointerException`
- **NumberFormatException:**
 - `String s="abc";`
`int i=Integer.parseInt(s);//NumberFormatException`
- **ArrayIndexOutOfBoundsException:** `int a[]=new int[5];`
 - `a[10]=50; //ArrayIndexOutOfBoundsException`

Cntd..

- Errors:
- Error is irrecoverable.

Example: `outOfMemoryError`, `VirtualMachineError` etc...

- Java Exception handling Keywords:
 - Try: try block consist of code that might throw exception and it must be fallowed by either catch or finally block.

- Syntax try

```
try {
```

```
//code that may through exception
```

```
– } catch(ExceptionClassName refObj)
```

```
try {
```

```
//code that may through exception
```

```
} Finally { }
```

Cntd..

- Java Exception handling Keywords:
 - Catch: catch block is used to handle the exception which is thrown by try block
 - We can include multiple catch block.
 - Syntax try
 - {
 - //code that may throw exception
 - } `catch(ExceptionClassName refObj)`

- Example:

```
Public class test1
{
public static void main(String args[])
{
    Int j=20,k=20;

    int i = 18/0; // throw exception
    Sytem.out.println("excute rest
code"+(j+k)):
}
}
```

```
Public class test1
{
public static void main(String args[])
{
    Int j=20,k=20;

    try {
        int i = 18/0; // may throw exception
    }
    catch(ArithmeticException e)
    { Sytem.out.println(e); }

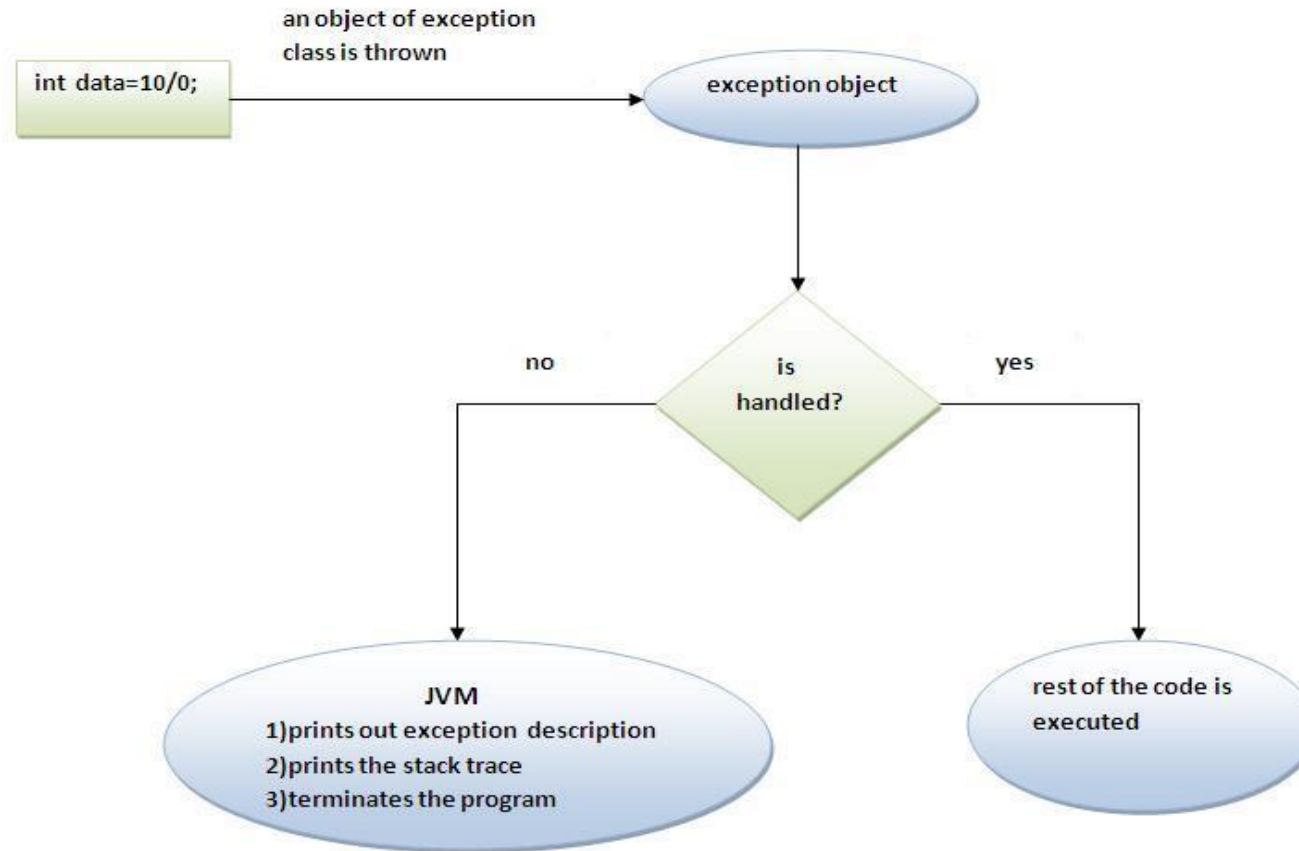
    Sytem.out.println("excute rest
code" + (j+k));
}
}
```

Cntd..

```
public class TestMultipleCatchBlock{  
  
    public static void main(String args[]){  
  
        try{  
  
            int a[]=new int[5];  a[6]=5;  
  
            a[5]=30/0;  
        }  
  
        catch(ArithmeticException e)  
        {System.out.println("task1 is completed");  
        }  
  
        catch(ArrayIndexOutOfBoundsException e){System.out.println("task 2 completed"  
);}  
  
        catch(Exception e){System.out.println("common task completed");}  
  
        System.out.println("rest of the code...");  
    }  
}
```

Handling Exceptions

- JVM firstly checks whether the exception is handled or not



Using multiple catch

- All catch blocks must be ordered from most specific to most general.

```
Public class test1
{
    public static void main(String args[])
    {
        try {
            int i = 18/0; // may throw exception
            int a[]=new int[5];
            a[6]=30/0;
        }
        catch(ArithmeticException e)
        { Sytem.out.println(e); }
        catch(ArrayIndexOutOfBoundsException e)
        {System.out.println("task2 completed");}
        Sytem.out.println("excute rest code");
    }
}
```

```
public class myexception{
    public static void main(String args[]){
        try{
            File f = new File("myfile");
            FileInputStream fis = new FileInputStream(f);
        }
        catch(FileNotFoundException ex){
            File f = new File("Available File");
            FileInputStream fis = new FileInputStream(f);
        }
        catch(IOException ex){
            //do something here
        }
        finally{
            // the finally block
        }
        //continue processing here.
    }
}
```

Nested TRY BLOCK:

```
try
{
    statement 1;
    statement 2;
    try
    {
        statement 1;
        statement 2;
    }
    catch(Exception e)
    {
    }
}
catch(Exception e)
{
}
```

- If a compatible match is found before an exact match, then the compatible match is preferred.
- We need to pay special attention on ordering of exceptions in the catch blocks, as it can lead to mismatching of exception and unreachable code.
- We need to arrange the exceptions from specific to general.

Finally Block

- Java finally block is a block that is used to execute important code such as closing connection, stream etc.
- Java finally block is always executed whether exception is handled or not.
- Java finally block must be followed by try or catch block.

```
class TestFinallyBlock{  
    public static void main(String args[]){  
        try{  
            int data=25/5;  
            System.out.println(data); }  
        catch(NullPointerException e){System.out.println(e);}  
        finally{System.out.println("finally block is always executed");}
```


Finally Block

- Java finally block is a block that is used to execute important code such as closing connection, stream etc.
- Java finally block is always executed whether exception is handled or not.
- java finally block must be followed by try or catch block.
- Finally block in java can be used to put "cleanup" code such as closing a file, closing connection etc.
- For each try block there can be zero or more catch blocks, but only one finally block.

```
try {  
    //code that may through exception  
} Finally { }
```

Cntd..

```
class TestFinallyBlock
{
Void a() throws NullPointerException,e2,e3
{String s="xyz";
int p= s/3;}

    public static void main(String args[])
    {
        try
        { a();
          int data=25/5;
          System.out.println(data);
        }
        catch(NullPointerException e){System.out.println(e);}
        Finally
        {
System.out.println("finally block is always executed"); }
        System.out.println("end of the program")
    }
```

Throw an exception

- The Java throw keyword is used to explicitly throw an exception.
- We can throw either checked or unchecked exception in java by throw keyword.
 - `throw new IOException("sorry device error");`
- Throws: Java throws keyword is used to declare an exception.
- It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.

Syntax for throws

```
return_type method_name() throws exception_class_name{  
    //method code  
}
```

throw

- Java throw keyword is used to explicitly throw an exception.
- We can throw either checked or unchecked exception in java by throw keyword. The throw keyword is mainly used to throw custom exception.

Syntax: throw new IOException("sorry device error");

- Java Exception propagation
- An exception is first thrown from the top of the stack and if it is not caught, it drops down the call stack to the previous method, and so on until they are caught or until they reach the very bottom of the call stack. This is called exception propagation.

throw	throws
Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
Throw is followed by an instance.	Throws is followed by class.
Throw is used within the method.	Throws is used with the method signature.
You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException.

User defined exception

```
Class Uexp extends Exception{
```

```
String s;
```

```
Uexp()
```

```
{s="Myexp";}
```

```
Uexp(String s)
```

```
{super(s);
```

```
this.s="my exception"}
```

```
    public String toString(){
```

```
        return (s+"The Exception is occurred") ;
```

```
    }}
```

```
Public class Example{
```

```
Public static Int i=10;
```

```
    public static void main(String[] args) {
```

```
        try {if(i==10)
```

```
            throw new Uexp("qwerty");
```

```
        }catch (Uexp ue) {
```

```
            System.out.println("Caught exception");
```

```
            System.out.println(ue);
```

```
        } }}
```